LangGraph Debugging — Worked Example

> Back to the exercise · Round 1: Basic · Round 2: Intermediate · Round 3: Real-world · Round 4: Multi-agent · Solution

One scenario, followed across all 4 rounds: Doc09's Build Task graph (the one that rebuilds Project 2's tool loop as nodes and edges) and its Project 4 descendant. Each round is a different way the graph's routing or state doesn't do what the diagram implies. Work through the rounds in order. Each ends with **"What do you think is wrong?"** — stop and actually answer before reading on.

- Round 1: Basic
- Round 2: Intermediate
- Round 3: Real-world
- Round 4: Multi-agent

Round: Basic

Setup:

```
def route(state):
    if state["needs_tool"]:
        return "call_tool"
    return "finish"

graph.add_conditional_edges("agent", route, {
    "call_tool": "tool_node",
    "done": END,
})
```

Symptoms: fails on the very first run where the agent decides it's actually done — not intermittent, reproducible every time that branch is taken.

Error output:

```
Traceback (most recent call last):
  File "project_3_documind_rag_agent/graph.py", line 44, in <module>
    result = compiled_graph.invoke({"needs_tool": False, ...})
```

```
  ...
KeyError: 'finish'
```

Expected vs. actual:

- Expected: when `route()` decides the agent is done, the graph moves to the end.
- Actual: LangGraph raises immediately with a bare `KeyError: 'finish'` — no node name, no hint, just the one word it couldn't find.

What do you think is wrong?

Round: Intermediate

Setup: routing is fixed. The graph's state includes a `findings` field that both a `search` node and a `calculate` node can update, since a task might need either or both:

```
class AgentState(TypedDict):
    task: str
    findings: dict

def search_node(state):
    results = run_search(state["task"])
    return {"findings": {"search": results}}

def calculate_node(state):
    result = run_calculation(state["task"])
    return {"findings": {"calc": result}}
```

Both nodes can run in the same graph run, one after the other, when a task needs both.

Symptoms: looks like "the model forgot what it found earlier" — a reasoning-layer problem — because the final answer only ever reflects whichever tool ran *last*.

Log output:

```
[search_node] state.findings = {'search': {'refund_policy': '30 days'}}
[calculate_node] state.findings = {'calc': 42}
[final_node] state.findings = {'calc': 42}
```

Expected vs. actual:

- Expected: after both nodes run, `state["findings"]` contains both the search results and the calculation result.

- Actual: whichever node ran second completely replaced `findings`, silently discarding whatever the first node had written.

What do you think is wrong?

Round: Real-world

Setup: state merging is fixed. The routing function decides whether to search again or finish:

```
def route(state):
    if state["confidence"] < 0.5:
        return "search_again"
    return "finish"
```

Symptoms: only sometimes happens — the great majority of real tasks finish normally within 2-3 search rounds. Every so often, a specific kind of ambiguous question causes the run to hit LangGraph's recursion limit instead of finishing.

Error output (only for certain inputs):

```
langgraph.errors.GraphRecursionError: Recursion limit of 25 reached without
hitting a stop condition. You can increase the limit by setting the
`recursion_limit` config key.
```

Expected vs. actual:

- Expected: the loop either finds a confident enough answer and finishes, or genuinely can't and says so clearly, within a small, predictable number of rounds.
- Actual: for most questions it finishes quickly. For a specific kind of question — one where the search results are consistently a little relevant but never quite enough to push confidence above 0.5 — it loops all the way to the hard recursion limit before failing, wasting 25 rounds of tool calls first.

What do you think is wrong?

Round: Multi-agent

Setup: the search loop above now has a real stopping rule. Separately, in Project 4, the Reviewer's conditional edge sends a draft back to the Writer for another pass whenever the Reviewer isn't satisfied:

```
def review_route(state):
    if state["review_passed"]:
        return "done"
    return "revise"

graph.add_conditional_edges("reviewer", review_route, {
    "done": END,
    "revise": "writer_agent",
})
```

Symptoms: testing the Writer and Reviewer as a 2-node pair, with a handful of normal draft/feedback examples, never hits a problem — a couple of revision rounds and it always converges. Running the full 4-agent pipeline on certain real tasks, the run doesn't converge and doesn't fail cleanly either — it just keeps going until something external (a cost alert, a timeout) stops it.

Log output:

```
[writer_agent] revision #1: draft submitted
[reviewer] REJECTED: missing the compliance detail from research_findings
[writer_agent] revision #2: draft submitted
[reviewer] REJECTED: missing the compliance detail from research_findings
[writer_agent] revision #3: draft submitted
[reviewer] REJECTED: missing the compliance detail from research_findings
... (continues past revision #20) ...
```

Expected vs. actual:

- Expected: the Writer/Reviewer loop either converges quickly or gives up after a small, fixed number of rounds with a clear "couldn't agree" result — Doc11's Build Task requires exactly this.
- Actual: the loop has no revision cap at all — `review_route` only ever checks `review_passed`, with no counter anywhere in state — so if the Reviewer is asking for something the Writer structurally can't produce (because, as it happens, that "compliance detail" was never actually written into `research_findings` by the Research agent to begin with), the two agents can disagree forever, and nothing stops them.

What do you think is wrong?

Full solution: Show me the solution